

Từ một lên hai, từ hai lên ba: dùng tư tưởng cải tiến thuật toán để giải các bài toán tăng quy mô và số chiều

Zhang Weida
Trường THPT số 1 Shaoguan, Quảng Đông

2005

Nguồn gốc: Dùng tư tưởng cải tiến thuật toán để giải các bài toán tăng quy mô và số chiều

IOI China candidate team papers / 2005 / Zhang Weida.doc

Tóm tắt

Ta thường gặp những bài toán đặc biệt được tạo ra bằng cách sửa đổi một bài toán đã biết: thêm một chiều, hoặc thêm một yếu tố, chẳng hạn từ một chiều sang hai chiều, từ hai chiều sang ba chiều. Bài viết gọi chung chúng là những bài toán có quy mô và số chiều tăng lên.

Để giải lớp bài toán này, có thể dùng tư tưởng cải tiến thuật toán. Phần đầu bài viết khái quát tư tưởng ấy. Phần hai bắt đầu từ một câu đố IQ thú vị để giới thiệu lối suy nghĩ và các con đường cải tiến cơ bản. Phần ba dùng nhiều ví dụ để minh họa cách cải tiến thuật toán. Phần cuối tổng kết lại tư tưởng cải tiến.

Từ khóa: tăng quy mô; cải tiến thuật toán; hạ chiều; phân tích; xây dựng.

1 Khái quát

Mỗi thí sinh học Olympic Tin học đều phải học một số thuật toán cơ bản trước khi áp dụng chúng vào bài tập. Nhưng đề bài có rất nhiều hình thức khác nhau, nên các thuật toán ấy thường không thể dùng máy móc. Đôi khi ta phân tích và thấy một bài toán có vẻ giải được bằng một phương pháp quen thuộc, nhưng bài toán lại khác bài đã gặp ở một điểm nào đó. Trường hợp được bàn trong bài này là: quy mô hoặc số chiều của bài toán lớn hơn bài gốc, chẳng hạn một chiều thành hai chiều, hai chiều thành ba chiều.

Ta có thể gọi đây là lớp bài toán tăng quy mô và số chiều. Khi giải chúng, thường cần quan sát đặc thù của bài toán, cải tiến thuật toán cũ, rồi mới giải được bài toán thực tế.

2 Dẫn nhập: từ một câu đố IQ

Đề bài. Trước khi vào nội dung chính, hãy xét một câu đố IQ, được kể là từng xuất hiện trong phỏng vấn tuyển dụng của IBM. Có hai cây nhang hoàn toàn giống nhau nhưng cháy không đều; mỗi cây cháy hết trong một giờ. Làm thế nào để xác định một khoảng thời gian đúng 45 phút?

Phân tích. Ta cần đo 45 phút, trong khi mỗi cây nhang cháy hết trong 60 phút. Có ba mô hình tự nhiên để suy nghĩ:

1. Không làm giảm thời gian cháy của mỗi cây; ghép hai cây lại để đo 45 phút.

2. Chỉ dùng một cây, làm giảm thời gian đo của nó xuống đúng 45 phút.
3. Làm giảm thời gian đo của cả hai cây, để tổng thời gian đo bằng 45 phút.

Mô hình thứ nhất hiển nhiên không thể. Mô hình thứ hai và thứ ba có điểm chung: ta phải làm giảm thời gian đo của một cây nhang. Nhưng giảm như thế nào, và giảm được tới đâu? Vì nhang cháy không đều, ta không thể đơn giản cắt nó ra. Tuy nhiên, nếu nghĩ tới dạng nhang cuộn, ta dễ nhận ra rằng có thể đốt cùng lúc cả hai đầu. Khi đó một cây cháy hết trong nửa giờ. Mô hình thứ hai vì vậy cũng bị loại, còn lại mô hình thứ ba.

Nhưng mô hình thứ ba không thể chỉ áp dụng trực tiếp cách “đốt hai đầu”, vì $0.5 + 0.5 = 1$ giờ. Ta cần cải tiến thêm: nếu một cây nhang đang cháy một đầu và còn lại thời gian t , thì khi đốt đầu còn lại, nó sẽ cháy hết sau $t/2$. Đặc biệt, khi $t = 60$ phút thì $t/2 = 30$ phút.

Từ đó có lời giải: cùng lúc đốt hai đầu của cây thứ nhất và một đầu của cây thứ hai. Khi cây thứ nhất cháy hết, đã qua 30 phút; cây thứ hai còn 30 phút. Lúc này đốt đầu còn lại của cây thứ hai; khi nó cháy hết thì đã qua thêm 15 phút. Tổng cộng ta đo được 45 phút.

Tiểu kết. Trong ví dụ trên, ta xuất phát từ việc xây dựng mô hình và cải tiến cách làm, rồi từng bước đi tới lời giải tốt hơn. Quy trình chính là:

1. Tìm lời giải ban đầu và các hướng cải tiến khả dĩ, tức ba mô hình trên.
2. Phân tích nguyên lý của cách làm: đốt hai đầu một cây nhang để đo nửa thời gian còn lại.
3. Cải tiến thuật toán. Trong quá trình này, thường không chỉ cải tiến hình thức thuật toán, mà phải dùng nội hàm và bản chất của nó để xây dựng một cách làm phù hợp với bài toán.
4. Giải bài toán.

Theo tác giả, có vài con đường thường dùng để cải tiến thuật toán:

- trực tiếp tăng quy mô của thuật toán;
- dùng liệt kê để xử lý phần quy mô tăng thêm;
- dùng tham lam để xử lý phần quy mô tăng thêm;
- phối hợp nhiều con đường trên.

Các phần sau sẽ dùng ví dụ để làm rõ mạch chính này.

3 Các con đường cải tiến thuật toán

3.1 Trực tiếp tăng quy mô thuật toán

Đối với bài toán đơn giản, đôi khi có thể xử lý trực tiếp, nhưng nhiều trường hợp vẫn cần nhìn ra một kỹ thuật ẩn bên trong.

3.1.1 Ví dụ 1: bài toán đường phố và mở rộng

Mô tả bài toán. Bài gốc: trong một lưới đường phố, tìm đường đi ngắn nhất từ góc dưới trái tới góc trên phải, mỗi bước chỉ được đi sang phải hoặc đi lên.

Bài mở rộng: trong cùng bản đồ ấy, tìm hai đường đi từ góc dưới trái tới góc trên phải sao cho không có cạnh nào xuất hiện trong cả hai đường, và tổng độ dài của hai đường là nhỏ nhất.

Phân tích. Bài gốc là một bài quy hoạch động điển hình. Nếu chia theo các đường chéo của lưới, đặt k là số bước đã đi và x_k là vị trí trên đường chéo thứ k , ta có thể viết một công thức dạng

$$f(k, x) = \min\{f(k-1, x) + d_{\text{right}}, f(k-1, x-1) + d_{\text{up}}\},$$

trong đó d_{right} và d_{up} là độ dài cạnh tương ứng.

Với bài mở rộng, nếu vẫn cố dùng cách “đơn giản” này một cách trực tiếp, hàm mục tiêu sẽ phải có bốn chỉ số, và điều kiện hai đường không được dùng chung cạnh cũng khó xử lý.

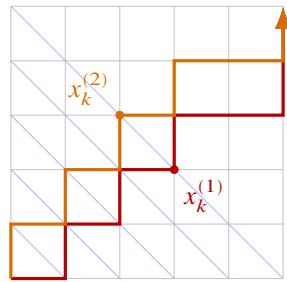
Ta quay lại bản chất của phương pháp ban đầu. Việc chia theo đường chéo thực ra biến bài toán thành một đồ thị nhiều tầng đặc biệt. Dùng biến quyết định $u_k = 0$ để biểu thị đi sang phải, $u_k = 1$ để biểu thị đi lên; trạng thái được cập nhật theo quyết định đó. Sau quan sát này, bài mở rộng trở nên tự nhiên: ta chỉ cần thêm một chiều trạng thái. Dùng

$$(x_k^{(1)}, x_k^{(2)})$$

để biểu thị vị trí của hai đường ở tầng k , và dùng

$$(u_k^{(1)}, u_k^{(2)})$$

để biểu thị hai quyết định tương ứng. Khi chuyển trạng thái, xét riêng hai đường, đồng thời loại bỏ những chuyển trạng thái làm hai đường dùng chung một cạnh.



Ở tầng k , trạng thái ghi đồng thời vị trí của hai đường; chuyển trạng thái bị loại nếu hai đường dùng chung một cạnh.

Figure 1: Mở rộng quy hoạch động một đường thành hai đường bằng cách thêm chiều trạng thái trên cùng các lớp đường chéo.

Tiểu kết. Ví dụ này cho thấy khi cải tiến, ta không nên chỉ bám vào hình thức của thuật toán cũ. Điều quan trọng hơn là phân tích bản chất của thuật toán ấy.

3.2 Dùng liệt kê để xử lý phân quy mô tăng thêm

3.2.1 Ví dụ 2: du lịch

Mô tả bài toán. Cho một ma trận số $N \times M$. Hãy tìm một hình chữ nhật con có tổng các số lớn nhất.

Phân tích. Nếu liệt kê mọi hình chữ nhật con rồi tính tổng, độ phức tạp là $\mathcal{O}(N^4)$, rõ ràng không chấp nhận được. Vì đây là bài toán hai chiều, ta thử hạ chiều.

Trước hết xét bài toán một chiều: trong dãy a_i , tìm đoạn con có tổng lớn nhất. Nếu làm thô sơ, ta có thể liệt kê đầu, liệt kê cuối, rồi cộng ở giữa, nhưng như vậy rất lãng phí. Dùng quy hoạch động, đặt f_i là tổng lớn nhất của một đoạn con kết thúc tại vị trí i , ta có

$$f_i = \max\{a_i, f_{i-1} + a_i\}.$$

Đáp án là $\max_i f_i$.

Bây giờ trở lại ma trận. Ta cần tìm quan hệ giữa ma trận và dãy. Ma trận có thể xem như nhiều dãy, nhưng các phần tử cùng cột ở những hàng khác nhau lại có liên hệ với nhau. Có thể thử xây dựng quy hoạch động đồng thời theo cả chiều trên–dưới và trái–phải, nhưng việc này không dễ. Với dữ liệu của bài, ta không nhất thiết phải tìm một thuật toán $\mathcal{O}(N^2)$. Cách tự nhiên là liệt kê hai biên trên và dưới, cộng các giá trị giữa hai biên theo từng cột để tạo thành một dãy một chiều, rồi dùng thuật toán đoạn con lớn nhất trên dãy ấy. Như vậy thu được thuật toán khoảng $\mathcal{O}(N^3)$.

Tiểu kết. Trong ví dụ này, ta dùng phương pháp đơn giản nhất là liệt kê để cải tiến thuật toán một chiều, từ đó giải bài toán có quy mô lớn hơn. Liệt kê tuy đơn giản, nhưng vẫn có thể giải quyết những bài toán khá phức tạp nếu đặt đúng chỗ.

3.2.2 Ví dụ 3: trận địa pháo binh

Mô tả bài toán. Bài toán NOI 2001 này cho một bản đồ lưới gồm địa hình đồng bằng và núi. Chỉ có thể đặt pháo binh trên ô đồng bằng. Một đơn vị pháo binh tấn công hai ô theo chiều ngang và hai ô theo chiều dọc. Cần đặt nhiều đơn vị nhất sao cho không đơn vị nào nằm trong phạm vi tấn công của đơn vị khác.

Phân tích. Xét bài đơn giản chỉ có một cột: có thể dùng tham lam đặt pháo càng lên trên càng tốt, hoặc viết truy hồi theo hàng.

Khi mở rộng sang nhiều cột, bài toán phức tạp hơn: không chỉ xét một cột có thể dùng lại giá trị của hàng trước hay không, mà còn phải xét đồng thời nhiều cột. Tuy nhiên dễ thấy rằng M nhỏ hơn nhiều so với N , tối đa chỉ 10. Hơn nữa, phạm vi tấn công của pháo khá lớn nên số cấu hình hợp lệ của một hàng không nhiều. Vì vậy có thể liệt kê các cấu hình hàng.

Ta dùng hai hàng làm một trạng thái. Gọi s_1 là cách bố trí của hàng trên và s_2 là cách bố trí của hàng dưới. Tất cả cấu hình hợp lệ của một hàng được liệt kê trước và đánh số. Nếu hàng thứ i không cho phép bố trí s_1 , hoặc hàng thứ $i + 1$ không cho phép bố trí s_2 , hoặc s_1, s_2 đặt pháo cùng một cột gây xung đột, thì trạng thái không hợp lệ. Ngược lại, ta chuyển từ một cấu hình hàng trước s sao cho s không xung đột với s_1 và s_2 :

$$F(i, s_1, s_2) = \max_s \{F(i - 1, s, s_1) + \text{cnt}(s_2)\}.$$

Ở cuối, lấy giá trị lớn nhất trong các trạng thái hợp lệ. Nếu R là số cấu hình hợp lệ của một hàng, độ phức tạp xấp xỉ $\mathcal{O}(R^3 N)$.

3.3 Dùng tham lam để xử lý phần quy mô tăng thêm

3.3.1 Ví dụ 4: luồng cực đại chi phí nhỏ nhất

Rõ ràng bài toán luồng cực đại chi phí nhỏ nhất có thể được cải tiến từ thuật toán luồng cực đại. Nói đơn giản, thuật toán luồng cực đại nhiều lần tìm một đường tăng từ nguồn tới đích, tăng luồng trên đường ấy, cho tới khi không còn đường tăng.

Vậy làm thế nào để xét thêm chi phí? Chi phí của mạng thường được định nghĩa là tổng, trên mọi cung, của chi phí cung nhân với lượng luồng trên cung đó; chi phí cung có thể âm. Luồng cực đại chi phí nhỏ nhất là luồng khả thi có giá trị cực đại và chi phí nhỏ nhất.

Không khó nhận ra rằng mỗi khi tăng thêm một đơn vị luồng, phần chi phí tăng thêm nên nhỏ nhất, hay chính xác hơn là nên làm giảm chi phí nhiều nhất nếu có thể. Vì vậy ta cải tiến thuật toán như sau:

mỗi lần chọn một đường tăng có chi phí nhỏ nhất, đặc biệt là đường có chi phí không dương nếu còn tồn tại, rồi tăng luồng theo đường đó. Khi không còn đường tăng cải thiện chi phí, ta thu được lời giải mong muốn. Ở đây, ý tưởng tham lam được dùng để đưa yếu tố chi phí vào bài toán luồng.

3.4 Phối hợp nhiều con đường

3.4.1 Ví dụ 5: Team Selection

Mô tả bài toán. Bài toán đến từ Balkan OI 2004 Day 1. Đội tuyển BP cần chọn thí sinh tốt nhất để dự IOI. Có N thí sinh rất mạnh, đánh số từ 1 tới N ,

$$3 \leq N \leq 500000.$$

Huấn luyện viên tổ chức ba cuộc thi và biết thứ hạng của từng thí sinh trong mỗi cuộc thi; không có hai thí sinh đồng hạng trong cùng một cuộc thi. Nếu thí sinh A xếp trước thí sinh B trong cả ba cuộc thi, ta nói A tốt hơn B . Nếu không có ai tốt hơn A , ta gọi A là *excellent*. Hãy tính số thí sinh excellent.

Ý tưởng ban đầu. Theo định nghĩa, cách đầu tiên là liệt kê từng thí sinh X , rồi liệt kê từng thí sinh Y để kiểm tra Y có tốt hơn X hay không. Việc kiểm tra chỉ cần so sánh ba thứ hạng, nhưng độ phức tạp là $\mathcal{O}(N^2)$, không đáp ứng giới hạn.

Một cải tiến nhỏ là duyệt X theo thứ hạng của cuộc thi thứ nhất. Khi xét X , chỉ cần liệt kê những Y xếp trước X ở cuộc thi thứ nhất, vì các thí sinh xếp sau chắc chắn không thể tốt hơn X . Cải tiến này mở rộng hướng suy nghĩ nhưng không làm thay đổi bản chất độ phức tạp.

Cải tiến tiếp theo: nếu Y không excellent vì có Z tốt hơn Y , thì khi xét X , nếu ta đã kiểm tra Z thì không cần kiểm tra Y . Bởi nếu Y tốt hơn X , thì Z cũng tốt hơn X . Do đó khi xét X , chỉ cần so sánh với tập các thí sinh excellent đã biết. Độ phức tạp giảm còn $\mathcal{O}(NK)$, với K là số thí sinh excellent, nhưng K vẫn có thể lớn.

Các cải tiến trên vẫn chủ yếu sửa khuyết điểm của thuật toán mô phỏng trực tiếp, chưa tận dụng đủ đặc thù của bài toán.

Tư tưởng hạ chiều. Bài toán gốc rõ ràng là từ hai chiều mở rộng lên ba chiều, nên ta xét trước trường hợp chỉ có hai cuộc thi.

Gọi A_i là thí sinh xếp thứ i ở cuộc thi thứ nhất, và $B[A_i]$ là thứ hạng của thí sinh ấy ở cuộc thi thứ hai. Duyệt các thí sinh theo thứ tự $A_1, A_2, \dots, A_N$. Với thí sinh A_i , mọi thí sinh A_j có $j < i$ đều đã xếp trước A_i ở cuộc thi thứ nhất. Do đó A_i không excellent khi và chỉ khi tồn tại $j < i$ sao cho

$$B[A_j] < B[A_i].$$

Ta chỉ cần duy trì

$$\text{best}_i = \min_{j < i} B[A_j],$$

rồi so sánh best_i với $B[A_i]$. Bài toán hai cuộc thi được giải trong $\mathcal{O}(N)$.

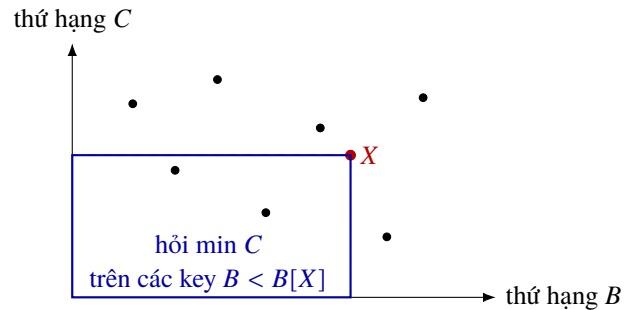
Điều quan trọng ở đây là không chỉ có công thức, mà còn phải hiểu bản chất của nó: khi duyệt theo cuộc thi thứ nhất, điều kiện thứ nhất đã được đảm bảo; ta chỉ cần giữ thông tin tốt nhất về cuộc thi thứ hai.

Mở rộng lên ba cuộc thi. Đặt $C[A_i]$ là thứ hạng của thí sinh A_i ở cuộc thi thứ ba. Khi xét $X = A_i$, ta cần biết có tồn tại $j < i$ sao cho

$$B[A_j] < B[A_i] \quad \text{và} \quad C[A_j] < C[A_i]$$

hay không. Nếu có, thì A_j tốt hơn A_i , nên A_i không excellent; nếu không, A_i là excellent.

Nếu vẫn liệt kê mọi $j < i$ thì độ phức tạp quay lại $\mathcal{O}(N^2)$. Ta cần tách hai điều kiện còn lại. Với điều kiện về cuộc thi thứ hai, dùng thứ hạng $B[A_j]$ làm khóa. Trong số mọi thí sinh đã xử lý có khóa nhỏ hơn $B[A_i]$, ta cần giá trị nhỏ nhất của $C[A_j]$. Nếu giá trị nhỏ nhất này nhỏ hơn $C[A_i]$, thì tồn tại một thí sinh tốt hơn A_i .



Duyệt theo cuộc thi thứ nhất; cây đoạn theo B lưu giá trị C nhỏ nhất.

Figure 2: Trong *Team Selection*, bài ba cuộc thi trở thành truy vấn tiền tố: đã xử lý theo hạng A , hỏi xem có điểm nào nằm trái và thấp hơn X trong mặt phẳng (B, C) hay không.

Như vậy cần một cấu trúc dữ liệu hỗ trợ hai thao tác:

- **chèn**: thêm một cặp (key, value);
- **hỏi**: trong các key nhỏ hơn một giá trị X , tìm value nhỏ nhất.

Ở đây key là thứ hạng trong cuộc thi thứ hai, còn value là thứ hạng trong cuộc thi thứ ba.

Cấu trúc phù hợp là cây đoạn hoặc cây tìm kiếm nhị phân có lưu giá trị nhỏ nhất trên mỗi đoạn key. Mỗi lá ứng với một key; mỗi nút trong lưu khoảng key của con nó và giá trị nhỏ nhất trong khoảng đó.

Khi chèn (a, b) , ta đi từ lá key a lên gốc, cập nhật mỗi nút bằng

$$\min(\text{giá trị hiện tại}, b).$$

Khi hỏi các key nhỏ hơn a , ta phân rã đoạn $[1, a - 1]$ thành các đoạn chuẩn trên cây và lấy min của các nút tương ứng. Cả chèn và hỏi đều mất $\mathcal{O}(\log N)$.

Thuật toán cuối cùng:

1. Duyệt thí sinh theo thứ tự từ cao xuống thấp trong cuộc thi thứ nhất.
2. Với thí sinh X , đặt key = $B[X]$ và value = $C[X]$.
3. Hỏi giá trị nhỏ nhất trong các key nhỏ hơn key.
4. Nếu giá trị đó nhỏ hơn value, có thí sinh tốt hơn X , nên X không excellent.
5. Ngược lại X là excellent.
6. Chèn (key, value) vào cây để các thí sinh sau xét tới.

Vòng duyệt có N bước, mỗi bước dùng một số thao tác $\mathcal{O}(\log N)$, nên tổng độ phức tạp là

$$\mathcal{O}(N \log N).$$

Tiểu kết. Trong ví dụ này, ta đã đi qua nhiều thuật toán: có thuật toán cơ bản, có những cải tiến chưa triệt để, rồi mới tới cách dùng cấu trúc dữ liệu. Sau khi có lời giải cho bài hai cuộc thi, ta không thể áp

dụng nó máy móc cho ba cuộc thi mà phải phân tích bản chất: điều kiện cuộc thi thứ nhất được bảo đảm bởi thứ tự duyệt, điều kiện cuộc thi thứ hai dùng làm khóa, còn điều kiện cuộc thi thứ ba trở thành giá trị cần tối ưu trên một đoạn khóa.

Tác giả chọn Team Selection làm ví dụ trọng tâm không chỉ vì nó cần quan sát nhanh và toàn diện, mà còn vì nó đòi hỏi hiểu biết nhất định về cấu trúc dữ liệu. Chính tác giả từng vấp ở điểm này khi làm bài.

4 Kết luận

Bài viết trình bày cách nhìn của tác giả về một lớp bài toán có quy mô và số chiều tăng lên. Do trình độ còn hạn chế, chắc chắn vẫn có những đạo lý chưa được hiểu thật thấu đáo, và một số thuật toán trong bài còn có thể cải tiến thêm. Nhưng mục đích chính của bài là nhấn mạnh rằng khi giải lớp bài toán này, ta cần kết hợp bản chất của thuật toán với đặc điểm cụ thể của bài toán.

Tư tưởng cải tiến thuật toán cũng là tinh thần khai phá và tiến thủ: không thỏa mãn với hiện trạng, mà phát triển thuật toán mới và cách nghĩ mới. Trong OI, nhiều bài toán không thể chỉ giải “cho xong”; ta cần liên tục cải tiến thuật toán và cấu trúc dữ liệu để thật sự giải quyết vấn đề.

Tư tưởng cải tiến thuật toán cũng là tư tưởng sáng tạo. Rất nhiều bài toán Olympic Tin học không thể áp dụng trực tiếp thuật toán kinh điển, mà cần thêm suy nghĩ sáng tạo và thực hành cụ thể. Trên thực tế, mọi thuật toán kinh điển đều được tổng kết từ những ý tưởng sáng tạo. Nếu ta dám sáng tạo, biết đâu một ngày nào đó cũng sẽ có một thuật toán kinh điển do chính chúng ta tạo ra.

Lời cảm ơn

Xin cảm ơn thầy Lin Shenghua đã hướng dẫn và bạn Yang Yi đã nhiệt tình giúp đỡ.

Tài liệu tham khảo

1. Jiang Wenzai chủ biên, *Con đường tới huy chương vàng: bồi dưỡng thi tin học trung học*, Nhà xuất bản Đại học Sư phạm Thiểm Tây, 2000.
2. Wu Yaobin, Cao Ligu, Xiang Qizhong, Zhu Quanmin, *Olympic kinh điển: giáo trình Olympic Tin học, nâng cao*, Nhà xuất bản Đại học Sư phạm Hồ Nam, 2001.
3. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa, 2003.
4. Đề bài và báo cáo lời giải do trang chính thức Balkan Olympiad in Informatics 2004 cung cấp.
5. Đề thi Olympic Tin học tỉnh Quảng Đông 2001.
6. Đề thi Olympic Tin học toàn quốc 2001.

A Bài du lịch

Mô tả bài toán. Đội GDOI đi chơi ở thị trấn Z . Thị trấn có $m + 1$ đường đông–tây và $n + 1$ đường nam–bắc, chia thành $m \times n$ khu vực. Danh thắng nằm trong các khu vực ấy. Khu vực ở hàng i từ trên xuống và cột j từ trái sang được gọi là $D(i, j)$. Đội GDOI cho trước điểm số $V(i, j)$ của mỗi khu vực, điểm có thể dương hoặc âm.

Để tiện tập hợp, các thành viên chỉ hoạt động trong một hình chữ nhật. Một phạm vi có thể biểu diễn bởi hai góc

$$(m_1, n_1), \quad (m_2, n_2),$$

với $m_1 \leq m_2$ và $n_1 \leq n_2$; nó gồm mọi khu vực $D(i, j)$ thỏa

$$m_1 \leq i \leq m_2, \quad n_1 \leq j \leq n_2.$$

Hãy tìm phạm vi có tổng điểm lớn nhất. Đội cũng có thể không đi đâu cả, chẳng hạn khi mọi phạm vi đều có tổng âm. Nếu có phạm vi tổng bằng 0, họ vẫn có thể đi.

Dữ liệu vào. Tập `travel.dat` có $m + 1$ dòng. Dòng đầu chứa m, n . Tiếp theo là m dòng, mỗi dòng có n số nguyên; số thứ j trên dòng thứ i là $V(i, j)$.

Dữ liệu ra. Tập `travel.out` gồm một dòng. Nếu đội đi trong một phạm vi $(m_1, n_1), (m_2, n_2)$, in tổng điểm của phạm vi đó. Nếu họ không muốn đi đâu, in N0. Không in dòng trống hoặc khoảng trắng thừa.

Ví dụ.

Input

```
4 5
1 -2 3 -4 5
6 7 8 9 10
-11 12 13 14 -15
16 17 18 19 20
```

Output

```
146
```

Input

```
2 3
-1 -2 -1
-4 -3 -6
```

Output

```
N0
```

B Trận địa pháo binh

Mô tả bài toán. Các tướng lĩnh muốn bố trí pháo binh trên một bản đồ lưới $N \times M$. Mỗi ô có thể là núi, ký hiệu H, hoặc đồng bằng, ký hiệu P. Chỉ có thể đặt pháo trên đồng bằng, và mỗi ô nhiều nhất đặt một đơn vị. Một đơn vị pháo tấn công hai ô gần nhất theo mỗi hướng ngang và dọc; địa hình không ảnh hưởng tới phạm vi tấn công.

Hãy bố trí nhiều đơn vị nhất sao cho không có hai đơn vị nào tấn công lẫn nhau.

Dữ liệu vào. Dòng đầu chứa N, M . Tiếp theo là N dòng, mỗi dòng là một chuỗi gồm P và H. Giới hạn:

$$N \leq 100, \quad M \leq 10.$$

Dữ liệu ra. In một số nguyên K , là số đơn vị pháo binh nhiều nhất có thể bố trí.

Ví dụ.

Input

5 4

PHPP

PPHH

PPPP

PHPP

PHHP

Output

6

C Team Selection

Mô tả bài toán. Interpeninsular Olympiad in Informatics sắp diễn ra, và đội Balkan Peninsula cần chọn các thí sinh tốt nhất. Có N thí sinh, đánh số từ 1 tới N , với $3 \leq N \leq 500000$. Ban huấn luyện tổ chức ba cuộc thi; mỗi thí sinh tham gia cả ba, và không có đồng hạng.

Thí sinh A được gọi là tốt hơn thí sinh B nếu A xếp trước B trong cả ba cuộc thi. Thí sinh A được gọi là excellent nếu không có thí sinh nào tốt hơn A . Hãy viết chương trình TEAM tính số thí sinh excellent.

Dữ liệu vào. Dữ liệu chuẩn gồm bốn dòng. Dòng đầu là N . Ba dòng tiếp theo là thứ hạng trong ba cuộc thi, mỗi dòng gồm các mã thí sinh theo thứ tự từ cao xuống thấp.

Dữ liệu ra. In một dòng gồm số thí sinh excellent.

Ví dụ 1.

Input

3

2 3 1

3 1 2

1 2 3

Output

3

Không có thí sinh nào tốt hơn thí sinh khác, nên cả ba đều excellent.

Ví dụ 2.

Input

10

2 5 3 8 10 7 1 6 9 4

1 2 3 4 5 6 7 8 9 10

3 8 7 10 5 4 1 2 6 9

Output

4

Các thí sinh excellent là 1, 2, 3, 5.